POSITION MANIPULATION

ADVANCED GAME DESIGN & DEVELOPMENT

DANNY JUGAN

VECTORS

- We know that every game object in Unity has a Transform component, which is divided into three parts: **Position**, **Rotation**, and **Scale**.

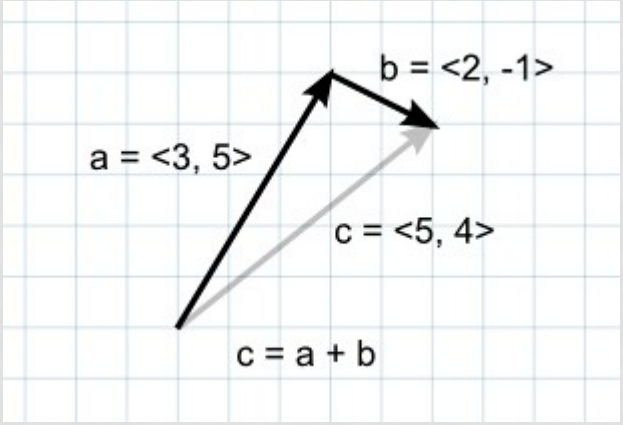
- **Position** (Vector3)
  - Points in space (x, y, z)

- In order to manipulate your world, you need to understand different arithmetic operations with Vectors.

VECTOR ADDITION

- **Addition**
- When two vectors are added together, the result is equivalent to taking the original vectors as "steps", one after the other.
- *Note that the order of the two parameters doesn't matter, since the result is the same either way.*

- What is a situation when you might be given Vector A and Vector B and need to find C?



VECTOR ADDITION

- Given the following piece of code, how would you find the position (Vector3) that is 5 units above and 3 units to the right of the supplied GameObject?

- `public GameObject ball;`
- `Vector3 newPos = ?`

- ```
  Vector3 newPos = ball.transform.position +
                   new Vector3(3, 5, 0);
  ```
  ```

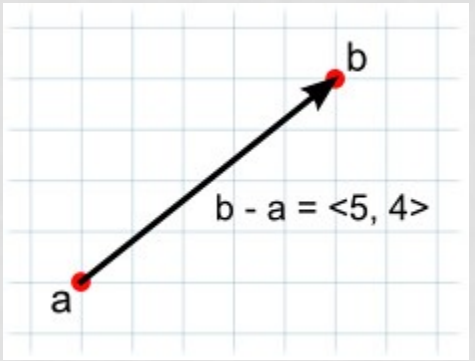
VECTOR ADDITION

- If the vectors represent forces then it is more intuitive to think of them in terms of their direction and magnitude (the magnitude indicates the size of the force).
- Adding two force vectors results in a new vector equivalent to the combination of the forces.
- This concept is often useful when applying forces with several separate components acting at once (e.g., a rocket being propelled forward may also be affected by a crosswind).

VECTOR SUBTRACTION

- **Subtraction**

- Vector subtraction is most often used to get the direction and distance from one object to another.



- The order of the two parameters **does** matter with subtraction.

VECTOR SUBTRACTION

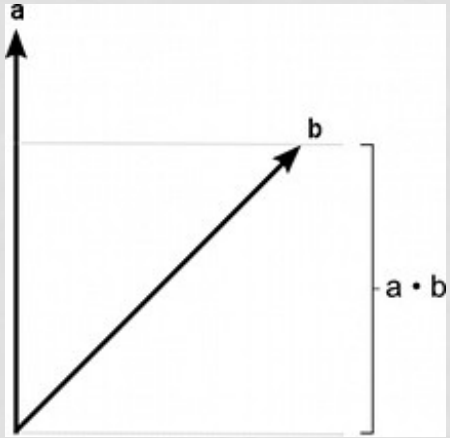
- Given the two Game Objects, complete the two lines of code that calculate vectors that point from the boy to the girl, and from the girl to the boy.

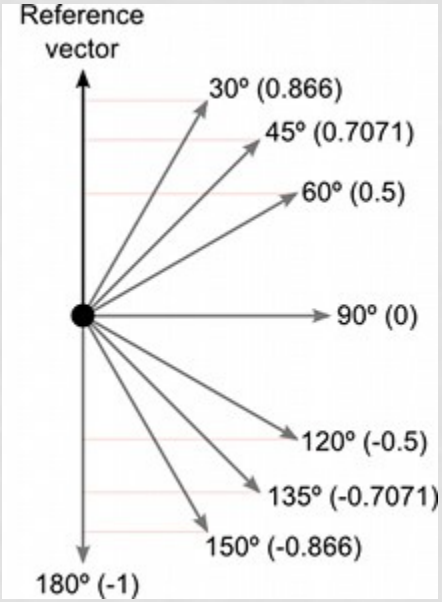
- `public GameObject boy, girl;`
- `Vector3 boyToGirl = ?`
- `Vector3 girlToBoy = ?`

- `Vector3 boyToGirl = girl.transform.position - boy.transform.position;`
- `Vector3 girlToBoy = boy.transform.position - girl.transform.position;`

DOT PRODUCT

- The dot product take two Vectors and returns a scalar (float) representing how far the first vector extends in the second vector's direction.





DOT PRODUCT

```
Vector3 forward = transform.forward;

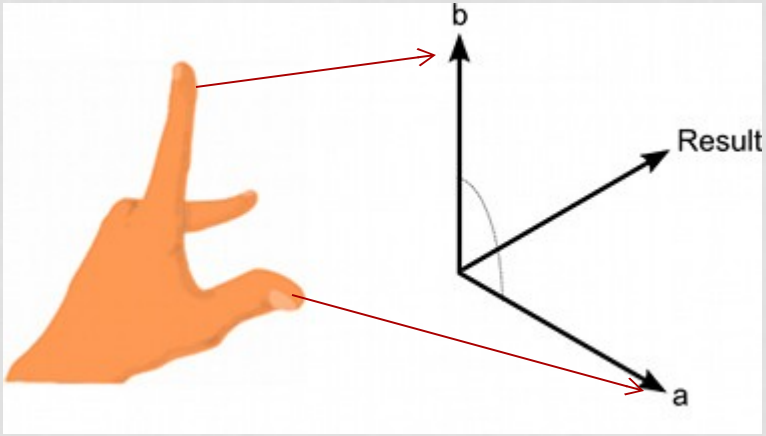
Vector3 toOther = other.position - transform.position;


if (Vector3.Dot(forward, toOther) < 0)

      print("The other transform is behind me!");
```

CROSS PRODUCT

- The Cross Product accepts two vectors and returns a vector.
- The resulting vector is perpendicular to the two input vectors.



NORMALIZED VECTORS

- A normalized vector is a vector that has a magnitude of 1.
- Normalized vectors are used when you care about the direction of a vector, and not its magnitude (length).
- There are built-in normalized vectors for each Game Object in unity.

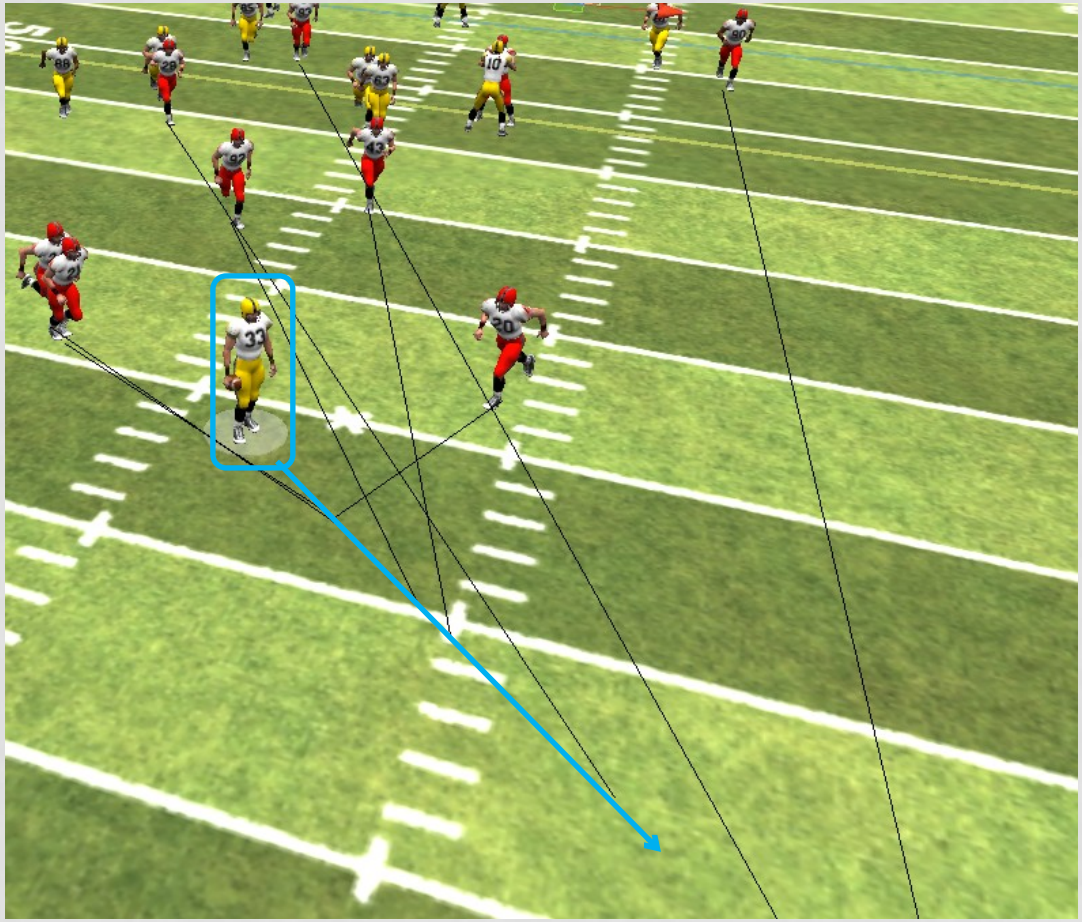
NORMALIZED VECTORS

- The lines in the image indicate built-in normalized vectors

- Black – `transform.up`
- Red – `transform.forward`
- Blue – `transform.right`

EXERCISE

- The character outlined in blue has the ball, and the red players are attempting to tackle him.
- The black lines show where they should run to take the best angle, but how is that calculated?



EXERCISE

Write the code to calculate and intelligent angle of pursuit for each player.

```
//Assume the following code script is on each red player's GameObject
//public GameObject ballHolder; <- player with ball


//Position on field where the player should run (where the black lines end)
private Vector3 target;

void Update(){

   target = ?

}
```

EXERCISE

Write the code to calculate and intelligent angle of pursuit for each player.

```
//Assume the following code script is on each red player's GameObject
//public GameObject ballHolder; <- player with ball


//Position on field where the player should run (where the black lines end)
private Vector3 target;

void Update(){

   float dist = Vector3.Distance (transform.position, ballHolder.transform.position);
   float scaleValue = 0.6f;
   target = ballHolder.transform.position +
           (ballHolder.transform.forward * dist * scaleValue );

}
```

*Note – You will need to adjust scaleValue depending on your world scale and how fast the players run. Larger values of this variable will project the point out farther.